



Metrics Collection with Prometheus

The Foundation of Observability for AI Infrastructure

A comprehensive guide for DevOps/SRE and ML infrastructure engineers working with Kubernetes and AI workloads

What Is Prometheus?

Metrics Monitoring System

Open-source time-series database with a pull-based approach that scrapes HTTP endpoints at /metrics

PromQL Query Language

Powerful, flexible query language that enables complex aggregations, alerting conditions, and visualizations

Cloud-Native Architecture

Designed specifically for Kubernetes environments with service discovery and dynamic configuration



Why Prometheus for AI Workloads?

Unified Observability

Collect, store, and query metrics from infrastructure, GPUs, models, and ML pipelines in a single system with consistent interfaces

High-Cardinality Support

Track detailed metrics with labels for individual models, nodes, and GPU instances to pinpoint performance bottlenecks

Rich Ecosystem Integration

Seamlessly works with DCGM for GPU metrics, Triton Inference Server, MLflow, and Kubernetes for complete visibility

Actionable Insights

Power alerts, dashboards, and automated scaling decisions with real-time metrics and historical data

Core Prometheus Concepts



Metrics

Numeric measurements collected at regular intervals, forming time-series data

Labels

Key-value pairs that add dimensions to metrics (gpu="0", model="bert")

PromQL

Query language enabling complex expressions, aggregations, and transformations

Scrape Targets

Applications exposing /metrics endpoints that Prometheus pulls from

Understanding Metric Types

Counter

Cumulative metrics that only increase (or reset to zero)

Example: `total_inferences_served` tracks the running count of inference requests handled

Gauge

Current value metrics that can increase or decrease

Example: `gpu_memory_used_bytes` shows the real-time memory consumption

Histogram

Distribution metrics bucketed by value ranges

Example: `inference_duration_seconds_bucket` for tracking latency distributions

Summary

Similar to histograms but calculate quantiles client-side

Example: `request_duration_quantiles` for p50/p95/p99 latency values

Exporters for AI Infrastructure

Critical Components for Complete Observability

Exporters bridge the gap between your AI infrastructure components and Prometheus by exposing metrics in the expected format.



Node Exporter

System-level metrics including CPU, memory, disk, and network statistics for host machines



DCGM Exporter

NVIDIA GPU metrics including utilization, memory usage, temperature, and power consumption



Custom Exporters

Application-specific KPIs from FastAPI, TorchServe, or custom ML services



Triton Server

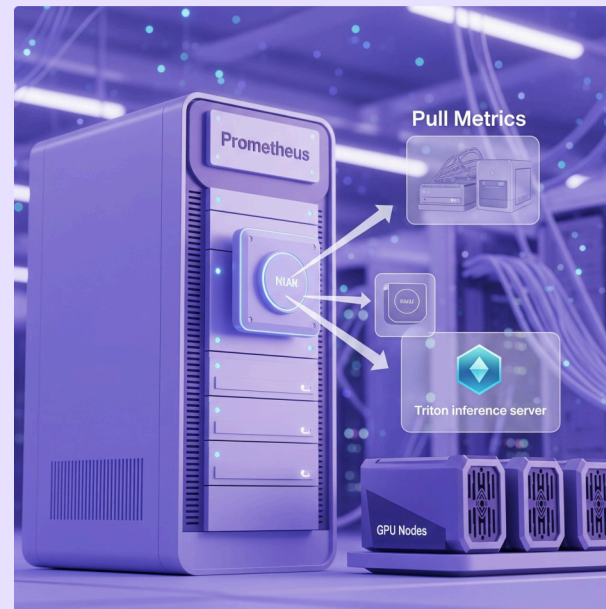
Model serving metrics including inference requests, latency, queue time, and cache hit rates

Scrape Configuration Example

prometheus.yml snippet:

```
scrape_configs:  
  - job_name: "gpu-metrics"  
    static_configs:  
      - targets: ["node1:9400", "node2:9400"]  
  
  - job_name: "triton"  
    static_configs:  
      - targets: ["triton-svc:8002"]
```

Prometheus periodically scrapes these targets, collecting all metrics from the specified endpoints. Each job creates a [job](#) label that helps organize and filter metrics.



Scrape intervals are configurable, with defaults typically set to 15 or 30 seconds for a balance of granularity and resource usage.

PromQL in Action

PromQL enables powerful querying capabilities for analyzing AI infrastructure performance

GPU Utilization (avg per cluster)

```
avg(DCGM_FI_DEV_GPU_UTIL)
```

Shows the average GPU utilization across all monitored GPUs in your cluster

Model Inference QPS (per model)

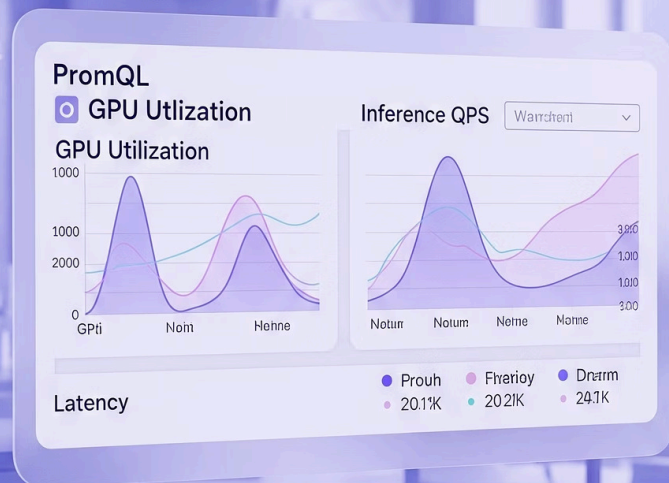
```
rate(nv_inference_count[1m]) by (model)
```

Calculates queries per second for each model over a 1-minute window

p95 Latency

```
histogram_quantile(0.95, rate(http_request_duration_seconds_bucket[5m]))
```

Computes the 95th percentile of request latency from histogram data



Integration with Kubernetes

Automated Discovery and Configuration

The Prometheus Operator simplifies deploying and managing Prometheus instances in Kubernetes clusters.

Pod Annotations for Auto-Discovery

annotations:

```
prometheus.io/scrape: "true"
```

prometheus.io/port: "8002"

These annotations tell Prometheus to automatically discover and scrape metrics from this pod.



Prometheus can drive [Horizontal Pod Autoscaling](#) and custom autoscalers based on metrics, enabling efficient resource utilization for ML workloads.

Metrics Storage & Retention



Balancing Performance and History

Local TSDB

High-performance storage optimized for recent data (days to weeks) with efficient compression

Remote Storage

Long-term retention via Cortex, Thanos, or Mimir for historical analysis and compliance

Model Performance History

Retain metrics to track model drift, degradation, and performance changes over time

For AI workloads, historical metrics provide crucial insights into model performance evolution and help establish baselines for alerting and capacity planning.

Best Practices for AI Metrics



Meaningful Labels

Tag metrics with model name, version, and team ownership to enable precise filtering and accountability



Percentile Monitoring

Track p50/p95/p99 latency distributions, not just averages, to catch performance outliers affecting user experience



Cardinality Management

Limit high-cardinality labels to prevent database explosion and performance issues in Prometheus



Security Controls

Implement authentication, authorization, and TLS for metrics endpoints to prevent unauthorized access

⚠ Performance Alert: Watch for the "cardinality explosion" problem when using too many label combinations. Each unique label set creates a new time series in Prometheus.

Key Takeaways

Comprehensive Observability

Prometheus serves as the backbone of AI infrastructure monitoring, unifying metrics across all components

Multi-layer Visibility

Collect and analyze metrics from infrastructure, GPUs, ML models, and applications in a single system

Powerful Querying

PromQL enables sophisticated analysis, SLO enforcement, and alert conditions for AI workloads

Ecosystem Integration

Seamlessly works with Grafana, Alertmanager, and Kubernetes for a complete observability stack



Next steps: Implement exporters for your specific AI stack and design dashboards to visualize key performance indicators.